SuperCopyback: Revisiting Copyback on Modern High-Performance NAND Flash-based SSDs

Dong Huang, Bo Ding, Wei Tong✉, Dan Feng✉

Key Laboratory of Information Storage System, Engineering Research Center for Data System and Technology, Ministry of Education
Wuhan National Laborotory of Optoelectronics, Huazhong University of Science and Technology
{haltz,boding,tongwei,dfeng}@hust.edu.cn, ✉Corresponding Authors

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133348

***Abstract*—NAND flash-based SSDs have emerged as a critical storage solution due to their exceptional performance and cost-effectiveness. However, the sequential write limitation of NAND flash blocks necessitates garbage collection (GC) to reclaim space occupied by stale data. Nevertheless, the extensive data migration involved in GC significantly impacts performance and Quality of Service (QoS) of SSDs. To mitigate this issue, copyback has been proposed as a means to accelerate GC by eliminating off-chip data movements. Specifically, copyback reads data into on-plane latches and immidiately re-writes it into another page on the same plane.**

However, in the case of modern high-performance SSDs, copyback is rarely utilized due to the following challenges: (1) Copyback operates at the page-level and thus fails to effectively reclaim invalid data within the context of subpage-level mapping; (2) Copyback eliminates off-chip data movements, preventing data pages from being read out for Redundant Array of Independent NAND (RAIN) parity computation, thereby compromising SSD reliability. In this study, we introduce SuperCopyback as a solution that efficiently addresses these issues for modern SSDs. Firstly, we propose a Multiple-Read-One-Write (MROW) copyback approach through lightweight latch circuit modifications to enable subpage-level copyback implementation; additionally, we propose an orchestrated GC method to effectively utilize MROW copyback. Furthermore, we present a novel copyback-based RAIN scheme that conceals data pages readout latency in write operations and relocates parities to support efficient copyback. The experimental results on both synthetic and real traces demonstrate that SuperCopyback achieves a performance comparable to an ideal scenario where data movement of 4KB takes only 1ns.

***Index Terms*—NAND Flash Memory, Solid State Disk, Copyback.**

I. Introduction

NAND flash-based Solid-State Disks (SSDs) have emerged as the predominant storage medium for many applications due to their significantly superior performance compared to hard disks, primarily attributed to the inherent abundant parallelism [1]–[4]. Typically, an SSD comprises multiple channels, each of which is shared by multiple chips. Each chip consists of multi-planes and multi-plane commands can be used for simultaneous read/write operations on all planes [19]–[23]. Multiple blocks from different chips are organized into a superblock, with pages at identical offsets from these blocks grouped as superpages. The utilization of superpage-based management not only maximizes the exploitation of abundant parallelism and simplifies data management, but also facilitates the cross-chip Redundant Array of Independent NAND (RAIN) stripe organization, which significantly enhances the reliability of SSDs [5]–[8].

Due to the inherent sequential write constraint of flash blocks, Garbage Collection (GC) is essential for reclaiming space occupied by invalid data in order to accommodate incoming writes [9]–[11]. Specifically, GC entails the selection of a candidate block, migration of valid data to another available free block, and subsequent erasure of the candidate block. The performance, endurance, and Quality of Service (QoS) of SSDs are significantly influenced by GC due to its extensive data migration.

Copyback is an effective strategy for accelerating garbage collection (GC) by eliminating off-chip data movements through channels. Specifically, copyback reads data to on-plane data latches (i.e., page register) and then immidiately re-writes it to another page on the same plane. The effectiveness of copybacks have been validated by previous research [9], [10], [12]. This optimization significantly enhances GC performance, particularly in scenarios where a channel is shared by multiple high-performance NAND chips.

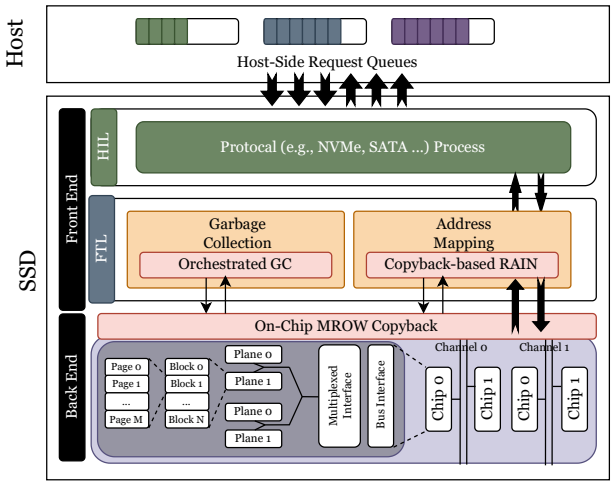


Fig. 1: Overview of SSD Archiecture and SuperCopyback

However, modern SSDs present notable challenges when employing copyback. ***Challenge #1 partial-valid page copyback:*** In the context of modern SSDs, pages typically possess a larger size (e.g., 16KB) compared to the host's I/O granularity of 4KB. To mitigate the resulting write amplification caused by this mismatch, SSDs employ subpage-level mapping, resulting in partial-validity within a page where certain subpages are valid while others remain invalid [13], [14]. As copyback operations occur at the page level, SSDs exclusively programme invalid data or resort to normal GC. ***Challenge #2 Parity Computation:*** Modern SSDs employ RAIN as a means to ensure data reliability in cases where ECC fails to correct errors [15], [16] or die-level failures occur [6], [8], thereby mitigating potential risks. Since the parity write operation requires XOR computation on other data pages in the stripe, the RAIN scheme cannot be used during copyback due to the elimination of off-chip data movements. This dilemma presents a tradeoff between performance and reliability.

In this paper, we propose ***SuperCopyback*** to tackle the challenges. As shown in Figure 1, SuperCopyback includes an efficient subpage-level copyback called ***Multiple-Read-One-Write (MROW) copyback***. MROW copyback is implemented via lightweight hardware modifications (i.e., a few transistors used to act as switches). The core of MROW is that multiple partial-valid pages can be merged into a full-valid page on the chip. We also propose an ***orchestrated GC scheme*** implmented in GC unit to use MROW copyback efficiently. Furthermore, SuperCopyback includes a novel ***copyback-based RAIN scheme*** implemented in address mapping unit. Copyback-based RAIN scheme relocates parities to avoid copyback stalls and hide the data readout latency in write operations. Our experimental results on both synthetic and real traces demonstrate that compared to an SSD with GC using off-chip data movements, SuperCopyback can achieve up to 1.346x throughput (1.263x on average), meanwhile significantly enhancing QoS by reducing 28.4% 99-th percentile latency on average, which is comparable to an ideal

scenario where data movement of 4KB takes only 1ns. In summary, we make the following contributions in this paper:

- We highlight the challenges associated with implementing copyback operations in modern SSDs, including limitations in the context of subpage-level mapping and potential conflicts with RAIN scheme.
- We propose SuperCopyback, which introduces a subpage-level copyback operation called MROW copyback, implemented through lightweight hardware modifications. Additionally, we present an efficient approach for utilizing MROW copybacks. Furthermore, we propose a novel copyback-based RAIN scheme to enable RAIN functionality within the context of MROW copybacks.
- We conducted comprehensive experiments to demonstrate the advantages of SuperCopyback. The experimental results reveal that SuperCopyback increases up to 34.6% throughput and reduces 28.4% 99-th percentile latency on average, compared to an SSD with GC using off-chip data movements.

II. Background

A. SSD Overview

Typically, an SSD is structured into three layers, as depicted in Figure 1 [1], [17]. The front-end encompasses host-interface logic (HIL) and flash translation layer (FTL). HIL receives and responds to requests from the host based on protocols (e.g., NVMe, SATA). FTL operates on a microprocessor and processes I/O requests while sending/receiving flash operations to/from the backend. FTL serves as the core of SSD firmware and is responsible for address mapping, garbage collection, wear leveling, etc. The backend consists of a hierarchically organized NAND flash array with multiple channels which transfer data between the front-end and back-end. Each channel is connected to multiple flash chips, each composed of multiple planes. A plane comprises numerous NAND blocks, with each block containing many pages. Page is the minimum read and program unit while block is the minimum unit of erase. Pages in a block must be sequentially programmed, causing GC problem.

B. GC and Copyback

Due to the sequential programming nature of NAND blocks, FTL typically employs a method of data writes by appending data to the block and recording the page location in a mapping table. Stale data pages are marked as invalid, and to reclaim these pages, FTL performs GC that involves migrating valid data from one superblock (referred to as victim) to another superblock before erasing the victim superblock. A physical page not only carries data but also metadata including Logical Page Address (LPA) and ECC (Error Correction Code) parity. During GC, the controller reads both the data and metadata, corrects errors using ECC parity, allocates a new address to LPA and updates the mapping table; finally, it writes the corrected data into NAND flash memory.

The impact of GC on I/O performance is widely acknowledged [9]–[11], [18], prompting NAND flash vendors to propose the use of copyback as a means to accelerate GC. Copyback involves reading a page to the plane register and immediately re-writing it on the same plane, thereby eliminating off-chip data movement through channels, as shown in Figure 2. While copyback holds significant potential, its current applicability in modern SSDs is limited, as discussed in Section III.

C. Characteristics of Modern NAND Flash-based SSD

Modern high-performance NAND Flash-based SSDs typically exhibit abundant parallelism through the utilization of multiple channels, chips, and planes. In order to fully harness this parallelism, SSDs commonly employ superpage-based data management. This approach involves grouping blocks at identical offsets from each plane to form a superblock, and then further grouping pages at the identical offset from each block to form a superpage. Allocation of pages from superpages is carried out in a round-robin manner. Superpage-based management can maximize the utilization of parallelism while also simplifying data management [5]–[8]. To fully exploit plane-level parallelism, independent multi-plane read is proposed to read multiple pages simultaneously from different offsets on planes [19]–[21]; for writes, buffer-based multi-plane write is proposed [22], [23]. In this paper, these approaches are also used to accelerate SSD performance.

Furthermore, superpage also facilitates the generation of RAIN parity, a technique commonly employed to enhance SSD reliability [6], [8], [24], [25]. Similar to RAID, RAIN organizes data pages into stripes and calculates a parity for each stripe using XOR. In the event of data page corruption due to ECC failure or chip-level failure, RAIN reads all other pages in the stripe and performs an XOR operation to recover the corrupted data page. A single superpage can be divided into one or multiple stripes. As flash block does not allow in-place updates, once generated, the parity is never updated, bringing in minimal overhead. Currently, enterprise-level SSDs typically employ a RAID-5 style RAIN to achieve full read performance [8], [24], [25], which is also used in this paper.

In addition, modern SSDs typically use 16KB pages, representing a fourfold increase from the 4KB pages employed for host I/O requests. This disparity results in significant write amplification and performance degradation. To address this issue, state-of-the-art high-performance SSDs implement subpage-level mapping [13], [14], [26]. Subpages are aggregated in the buffer and subsequently written to NAND memory once they can form a complete physical page. In our study, we adopt 4KB subpages while maintaining physical pages at 16KB.

III. Motivation

Modern solid-state drives (SSDs) commonly employ subpage-level mapping and superpage-based management techniques to optimize performance and ensure reliability. However, incorporating copyback into such SSDs presents unique challenges. In this section, we discuss these challenges and review the existing works.

A. Challenges

As shown in Figure 2, the first challenge arises from the limitation that conventional copyback operations can only be performed at a page level and cannot handle partial-valid pages effectively. While recent advancements like SOML enable reading of subpages across different pages within a plane, they are constrained by block decoder limitations preventing target locations from sharing same block [27]. As a result, these techniques cannot be utilized for GC reads. If invalid subpages are skipped, this would result in accelerated depletion of available storage space, leading to increased write amplification and diminished overall performance and lifespan of SSDs.

Second, as RAIN parity necessitates a XOR operation on all data pages within a cross-chip stripe, the use of copyback inevitably compromises reliability since it eliminates off-chip data transfer. Consequently, a trade-off between performance and reliability arises with the coexistence of either copyback or RAIN.

B. Existing Works

Copyback can accelerates data movement during GC. However, traditional copyback methods are unable to rectify bit errors. To address this issue, FastGC [10] and rcFTL [9] proposed a threshold-based copyback scheme to prevent uncorrectable errors. This strategy is also employed in this paper. The limitations in their design is

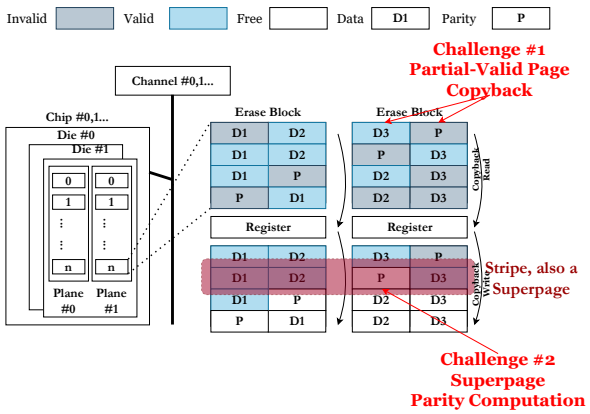


Fig. 2: The challenges in superpage-based SSDs.

that they are for conventional SSDs without superpage management, subpage-level mapping and RAIN protection, making them inadequate to address the aforementioned challenges. DecoupledSSD [18] interconnects channels to mitigate DRAM bus contention, although it still necessitates off-chip data movements during GC. Other interconnection-based approaches [28], [29] address channel contention by enhancing path diversity but at the expense of substantial architectural and hardware modifications. Besides, these approaches are based on conventional SSDs and therefore cannot effectively address the unique challenges posed by superpage-enabled SSDs. For instance, in superpage-enabled SSDs, garbage collection is executed simultaneously across all chips, which presents a significant challenge for path reservation in the interconnect network. Additionally, due to the absence of an intermediate buffer, the target chip's register must be idle to accept data; otherwise, the source chip can only wait, resulting in a sacrifice of chip-level parallelism. Furthermore, even if a dedicated path is always available, there still exists latency in data transfer through the links. These difficulties make using interconnect network during GC for superpage-enabled SSDs challenging. Hence, we focus on using copyback to accelerate GC from another perspective.

To the best of our knowledge, SuperCopyback represents a pioneering effort in enabling efficient copyback on modern SSDs to mitigate the detrimental impact of off-chip data movements during GC. We believe that this makes SuperCopyback novel and a significant contribution to the field.

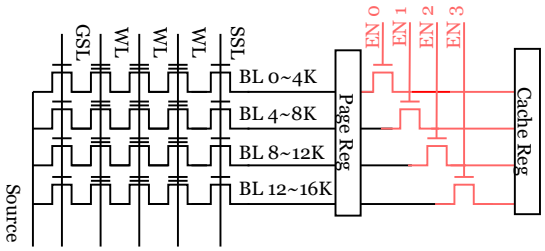
IV. Design

A. Overview

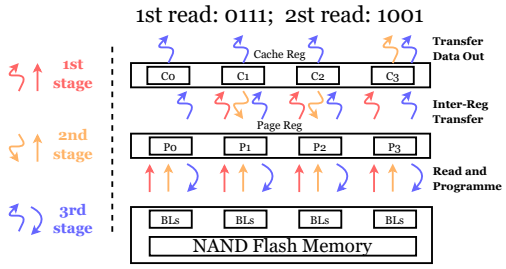
SuperCopyback enables efficient copyback on modern SSDs, as shown in Figure 1. First, it implments on-chip MROW copyback by lightweight hardware modifications. MROW copyback supports not only full-valid page copyback but also partial-valid page copyback. Then, an orchestrated GC is implemented in GC unit to exploit MROW copyback to accelerate GC. In the end, to ensure sufficient reliability when using copyback, a copyback-based RAIN scheme is implemented in the address mapping unit, which hides the data readout latency in program operations during MROW copyback and relocates parity pages to avoid copyback stalls.

B. Subpage Copyback

We propose a subpage-level copyback called Multiple-Read-One-Write (MROW) copyback. The core idea of MROW is that it first uses independent multi-plane read to read pages from all planes to the plane registers. Then, MROW performs a second read for each



(a) Subpage-level inter-register transfer.



(b) Steps of a MROW copyback.

Fig. 3: Implementation of MROW copyback.

plane to read another page to replace the invalid subpages in the first read page.

Two-Latches Meachanism: To enable on-chip merge of read data, we implement a lightweight subpage-level inter-register transfer. We exploit the existing two-latches mechanism in flash chip [3], [30]. Conventionally, a bit-line is connected to a data latch; and a data latch is connected to a cache latch. In a read operation, data bits are sensed through bitlines to data latches; and then transferred to cache latch if cache latches are empty. If possible, channel controller transfers data out from cache latches; meanwhile, chip can still read bits into data latches, accelerating reads through a pipelined style. The writes can also be accelerated in a similar style.

Subpage-level Inter-Register Transfer: To implement subpage-level inter-register transfer, we divide the bitlines, data latches and cache latches in a plane into multiple groups at subpage granularity, as shown in Figure 3a. For each group, we add a transistor and connect it to the *EN* inputs of all cache latches. Then, for target groups, we enable the transistor so the cache latches can store bits from data latches; for other groups, we disable the transistor so the corresponding cache latches can hold the original bits. Similarly, another four transistors can be connected to data latches to enable a symmetric cache-to-data subpage-level inter-register transfer. Since a latch typically only takes a few cycles to store a bit, the latency overhead is negligible. With subpage-level inter-register data transfer, we can read two partial-valid pages, merge the valid subpages into one full-valid page and programme it into NAND cells. If two pages have valid subpages at the same offset (we call it conflict subpages), we read out the one in the first read and write it as the normal GC such as the read out latency can be hided by the second read. MROW can also support full-valid page read by eliminating the first read.

Metadata Readout: To update the mapping table, the metadata of pages must be transferred out. Fortunately, we observe that page program latency is significantly higher than the data transfer latency. Hence, hiding data transfer latency during program operations is possible. Specifically, the chip awaits for the end of reads in copyback

(two reads for partial-valid pages or one read for full-valid pages) and the end of conflict subpages readout (see the example below for details); then, the controller sends copyback write command to the chip; simultaneously, the chip informs the controller that the data on cache register is ready for readout.

Command Support: The existing copyback commands can be leveraged to support the MROW copyback by adding four bits to each plane address in the command. These bits are used to indicate the validity of subpages. Hence, the inter-register transfer can be properly enabled to implement MROW copyback.

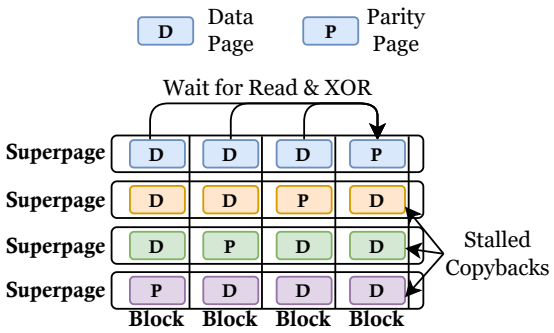
Illustrative Example: Figure 3b illustrates an example, where a page has four subpages. Assume the first read page has three valid subpages (the last three ones, tagged as 0111) and the second read page has two valid subpages (the first and last one, tagged as 1001). The page register and cache register are divided into four groups (P0 P4, C0 C4). In the first stage, the controller sends the first copyback read command to read the first page into page register same as a normal read operation; when read ends, the last three valid subpages are transferred from page register (P1-P3) to cache register (C1-C3). In the second stage, the controller sends the second copyback read command and readout data from C3 if possible. Once read operation ends, valid data are transferred from C1, C2 to P1, P2 to form a full-valid page. In the third stage, all data are copied from page register to cache register; then the data in page register are programmed into NAND cells and the data in cache register are transferred out when channel becomes idle. Notice that for a full-valid page, the first read is eliminated and MROW can be simplified by starting from the second stage.

C. Orchestrated GC Based on Subpage Copyback

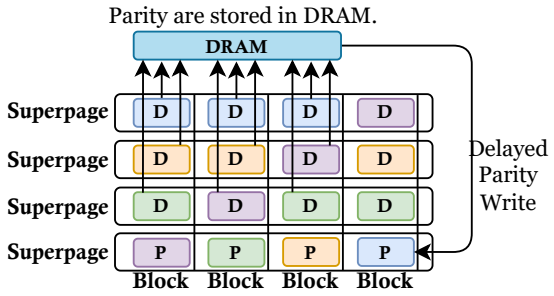
We employ a greedy algorithm to orchestrate copybacks in GC, aiming for the fewest readouts in the second stage. It is important to note that the copyback write operation must wait for the completion of readouts in the second stage. Given that read latency is comparable to data transfer latency, it is not always possible for read operations to effectively conceal data transfer latency. This motivates us to seek ways to minimize readout occurrences in the second stage of MROW copyback.

In order to adhere to the maximum number of successive copybacks, we record the executed copyback counts as page metadata and implement a threshold-based copyback scheme, similar to previous studies. Put simply, if the copyback counts exceed the pre-defined threshold, the subpage is migrated via conventional GC. Additionally, we use one bit in DRAM to record the copyback feasibility of each subpage (1 indicating that this subpage can be migrated through copyback), allowing for indication of the subpages' copyback feasibility in a page with a 4-bits bitmap. As the copyback count metadata is read during GC, the copyback feasibility bit can be updated by comparing it with the copyback counts against the threshold. Prior to performing GC, all valid pages of a plane are grouped into 16 lists corresponding to various 4-bits bitmap values.

For the target chip, we select either one full-valid page or two partial-valid pages on each plane to execute MROW copybacks. To begin, we iterate through the list starting with a bitmap value of *1111* (representing full-valid pages in copyback). If this is not feasible, we search for viable matches from the list *0001* to *1110*. Specifically, for a list with a bitmap of *abcd*, we can identify its corresponding optimal matches (i.e., no readout required in the second stage) within the list *1111-abcd*. In case the list *1111-abcd* is empty, we explore other lists with complementary bitmaps *a'b'c'd'* (i.e., where *a'b'c'd' or abcd = 1111*). If such a match cannot be found, normal GC procedures are employed as fallback.



(a) Conventional RAIN layout based on RAID-5.



(b) Copyback-based RAIN layout.

Fig. 4: Comparison between conventional RAIN and copyback-based RAIN.

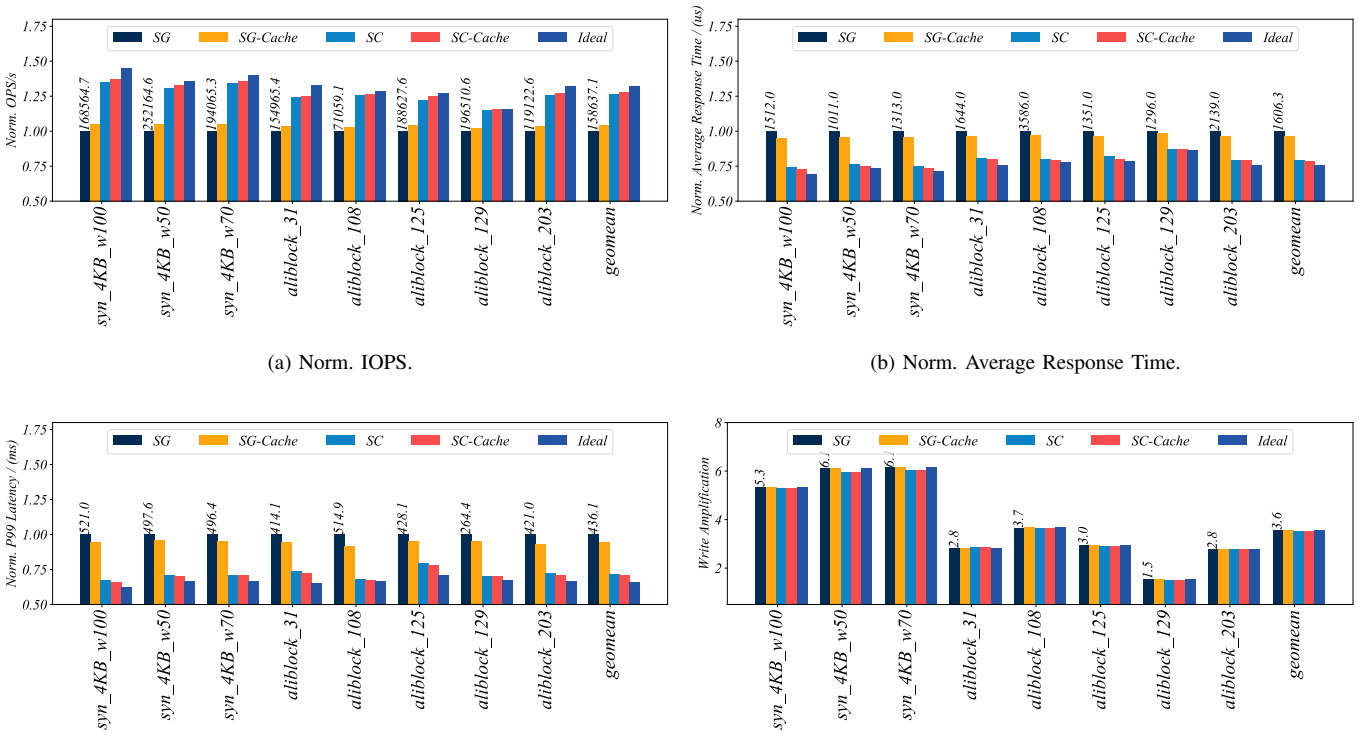
In addition, to maintain superpage management after GC operations, we impose an upper limit on the maximum number of copybacks per chip, which is $\frac{N_{validsubpages}}{Size_{superpage}}$. Once this limit is reached, GC on that chip reverts back to normal mode. Such a limit can also prevent potential delays in GC due to an imbalance in the distribution of valid subpages across different chips. While rare, this scenario remains a possibility and warrants consideration.

The subpage in the second read, if valid but not feasible for copyback, may overwrite the corresponding subpage in the first read designated for copyback. To prevent such exceptional cases, we ensure that the copyback feasibility bitmap matches the validity bitmap (i.e., all valid subpages in the second read are feasible for copyback). Additionally, in order to fully exploit plane-level parallelism through multi-plane operations, if no feasible match is identified on any plane, the corresponding chip also reverts to normal GC. It is worth noting that independent multi-plane read, a prevalent technique in modern NAND chips [19], [20], [31], is used to eliminate the identical page offset requirement in a multi-plane read. This enables full utilization of plane-level parallelism during copyback reads and eliminates the need to align read pages on all planes during MROW copyback. Multi-plane program during copyback is also used as conventional superpage-enabled SSDs.

D. Parity Computation and Storage

In this section, we elaborate on the implementation of efficient RAIN schemem in copyback-based GC.

The computation of parity, which involves XOR computation on all data pages in the stripe, necessitates the controller to read both metadata and data during copybacks. We can observe that the latency associated with these transfers is considerably lower than



(a) Norm. IOPS.

(b) Norm. Average Response Time.

(c) Norm. 99-th Percentile Latency.

(d) Write Amplification.

Fig. 5: Performance Comparison between Different SSDs. The results are normalized to SG.

program latency. To mitigate the impact of data transfer overhead, we strategically execute it during write operations. For a 1.6GB/s channel, transferring 64KB data consumes 40μs while program latency typically is higher than 300μs even for modern high-performance flash chips. Hence, we can effectively conceal this transfer latency by performing data transfers concurrently with program operations. As illustrated in Figure 3b, during the 3rd stage of MROW copyback, not only do we transfer page metadata but also the entire page data.

Furthermore, we propose an optimized RAIN layout to mitigate copyback stalls. In the conventional RAID-5 based RAIN layout, both data and parity are stored within the same superpage (refer to Figure 4 where different colors mean different stripes). However, during GC, the process of writing parity is delayed due to the necessity of reading and computing other data pages using XOR operation in the stripe. The subsequent data copyback are stalled since NAND blocks allow only sequential page writes. To address this challenge, our proposed approach relocates all parity pages to the end of NAND blocks.

As depicted in Figure 4, similar to conventional SSDs, data pages are read into DRAM and controller uses XOR to compute parity. However, instead of immediately writing the parity, we temporarily store it in DRAM until all data pages have been written. This strategy ensures that copybacks are not stalled by parity writes, enabling uninterrupted high-speed copyback operations. Notice that RAIN layout remains fixed, which means that once a page is corrupted, the controller can find other pages in its stripe based on the pre-defined layout. Besides, the parity page still resides in the same plane as original (i.e., only its page offset is changed). This eliminates needs for additional mapping and ensures recovery from page-level or chip-level failures as fast as original RAIN scheme.

E. Overhead

The SuperCopyback technique introduces a acceptable overhead. In the case of MROW, this overhead primarily arises from the modifications made to the control circuits of the data and cache latches, involving only a few transistors as a switch. Considering that each plane register comprises hundreds of thousands of data/cache latches, each composed of tens of transistors, the additional overhead is negligible.

In the case of orchestrated garbage collection (GC), the primary source of overhead arises from the DRAM utilization for an additional copyback feasibility bitmap. Considering that each subpage (4KB in this study) only requires one bit, the total memory overhead for a 1-TB SSD amounts to 32MB. It is worth noting that this bitmap can be stored at the end of the superblock and loaded into DRAM when this particular superblock is chosen as the victim for GC, similar to how a bitmap indicating page validity is stored in usual SSDs [13]. Consequently, if implemented in such a manner, the memory overhead becomes negligible (e.g., 64KB for a 2GB superblock). The main overhead of the proposed RAIN scheme arises from the buffer used for temporarily storing delayed parity writes. In our configuration, the memory overhead amounts to 64 MB ($\frac{2GB(superblocksize)}{32(stripesize)}$). For larger stripes, the memory consumption is reduced.

Typically, a 1-TB high-performance SSD (especially enterprise-level SSD) contains more than 1-GB DRAM, making the overall DRAM overhead acceptable from our perspective. It is worth noting that the consumed DRAM can also be substituted with cost-effective memory options such as PCM and SLC NAND or host memory through HMB technology to enhance cost-effectiveness. This substitution does not impact performance since PCM or SLC NAND still offer significantly faster speeds compared to TLC NAND, and parity/bitmap writes only constitute a minimal proportion of total I/O.

TABLE I: SSD Parameters

# of channels	4	# of chips	8
# of blocks	based on trace footprint	# of planes	4
RAIN Stripe	32	# of pages	1024
Channel BW	1.6GB/s	Page/Subpage Size	16KB/4KB
Read Latency	45 μs	Program Latency	320 μs
Block Erase Latency	3 ms	Over-Provision	15%

TABLE II: Workload Statistics

Trace	Requests	Average Size(KB)	Read Ratio (%)	Footprint (GB)
syn_4KB_w100	13107200	4	0	128
syn_4KB_w70	13107200	4	30	128
syn_4KB_w50	13107200	4	50	128
aliblock_31	3959045	9.22	0.29	54.6
aliblock_108	3249302	24.37	47.21	39.8
aliblock_125	8980886	9.21	19.07	58.2
aliblock_129	4285829	16.11	21.69	99.9
aliblock_203	4722134	12.40	2.25	30.4

In addition, in the event of a power failure, the bitmaps and parity can be reloaded or recomputed, eliminating the need for capacitors or batteries in DRAM to ensure persistence.

V. Evalution

A. Experimental Environment

We use MQSim, which is a widely-used simulator [17], to model a high-performance TLC SSDs. The detailed configuration of the SSD is listed as Table I. SSD performs GC when there is only one free superblock. We adjust the device capacity based on the footprint of each trace to avoid too large over-provison space. We extend MQSim to support subpage-level mapping and RAID-5 based RAIN scheme. We compare the following schemes: (1) SG, a superpage-based SSD using normal GC. (2) SG-Cache, an SG SSD with cache read/program using on-chip cache latches, (3) SC, an SSD using SuperCopyback that the maximum number of successive copybacks is 8 for GC, (4) SC-Cache, SC SSD with cache read/write for non-copyback IO, (4) Ideal, an SSD same as SG-Cache but its 4KiB data transfer overhead during GC is 1ns, which is used to emulate the ideal scenario.

We select both synthetic and real traces to evaluate the performance benefits. The synthetic traces are generated 4KiB uniform random workloads with different write ratios (50%, 70%, 100%) with 50GB size in total on a 128GB SSD. The real traces are widely-used block traces from Alibaba Cloud [32]. The characteristics of trace are listed as Table II. We ignore the I/O timestamps to make traces intensive. To avoid too much over-provision space, we adjust the device capacity based on trace foorptints.

B. Performance

In this section, we assess the performance advantages of SuperCopyback in terms of throughput, average response time, tail latency at the 99th percentile and write amplification (Figure 5). The following observations can be made. Firstly, in comparison to SG, SC achieves a maximum of 1.346x (34.6%) higher IOPS with an average improvement of 1.263x (26.3%); it reduces the average response time by up to 25.7% with an average reduction of 20.9%; and it decreases p99 latency by up to 32.8% with an average decrease of 28.4%. These results demonstrate that SuperCopyback not only significantly enhances throughput but also improves the QoS for SSDs. Furthermore, in comparison to SG-Cache, SC-Cache also demonstrates comparable performance benefits; similarly, when compared to SC, SC-Cache exhibits marginal performance advantages. This implies that

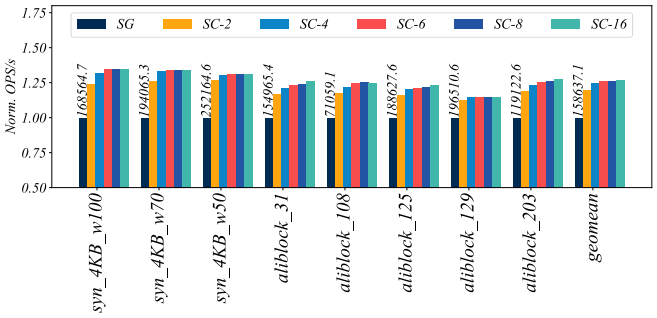


Fig. 6: The Impact of Copyback Threshold.

SuperCopyback can still derive advantages from cache read/write operations during the execution of user I/O requests; however, compared to cache read/write activities, SuperCopyback significantly enhances GC performance. Moreover, the SC-Cache achieves a performance close to Ideal (96.9% IOPS of Ideal), indicating that SuperCopyback effectively and efficiently mitigates the detrimental effect associated with data movements during GC. In the end, the write amplification of all schemes are nearly identical, indicating that SuperCopyback does not exert any adverse impact on the endurance of the SSD; the performance benefits of SuperCopyback also exhibit a positive correlation with write amplification, as higher write amplification leads to more severe GC and consequently increased data movements, thereby exacerbating its detrimental effects.

C. The Impact of Copyback Threshold

In this section, we assess the impact of varying copyback thresholds (i.e., the maximum number of consecutive copybacks for a subpage before ECC becomes unable to correct errors) on system performance. We explore copyback thresholds ranging from 2 to 16 (SC-2 to SC-16), and present the corresponding experimental results in Figure 6. The experimental results demonstrate that even with a copyback threshold of 2, SuperCopyback still exhibits significant performance benefits (1.198x IOPS). Furthermore, as the copyback threshold increases, the performance advantages of SuperCopyback also increase. When the copyback threshold reaches 6, SC achieves comparable performance to that obtained under higher copyback thresholds. This implies that in order to fully exploit SuperCopyback, it is only necessary for the ECC to ensure that at least 6 copybacks can be performed before ECC failures occur. We believe this is possible according to the results in prior works [9], [10].

VI. Conclusion

In this study, we propose SuperCopyback to mitigate the adverse impacts of off-chip data movements. SuperCopyback implements MROW copyback to support subpage-level copyback via lightweight hardware modifications; in addition, an orchestrated GC scheme is proposed to exploit MROW copyback. Furthermore, a copyback-based RAIN scheme is introduced to support efficient copyback, achieving both high reliability provided by RAIN and high performance provided by copyback. The experimental results on both real and synthetic traces demonstrate that SuperCopyback can significantly enhance the performance of modern SSDs.

VII. Acknowledgements

This work was supported by National Key R&D Program of China under Grant 2023YFB4502100 and also supported by the NSFC under Grant No.62172178, No.61821003.

REFERENCES

[1] Y. Hu, H. Jiang, D. Feng, L. Tian, H. Luo, and S. Zhang, "Performance impact and interplay of ssd parallelism through advanced commands, allocation strategy and data granularity," in *Proceedings of the international conference on Supercomputing*, 2011, pp. 96–107.

[2] H. Wan, X. Sun, Y. Cui, C.-L. Yang, T.-W. Kuo, and C. J. Xue, "Flashembedding: storing embedding tables in ssd for large-scale recommender systems," in *Proceedings of the 12th ACM SIGOPS Asia-Pacific Workshop on Systems*, 2021, pp. 9–16.

[3] J. Park, R. Azizi, G. F. Oliveira, M. Sadrosadati, R. Nadig, D. Novo, J. Gómez-Luna, M. Kim, and O. Mutlu, "Flash-cosmos: In-flash bulk bitwise operations using inherent computation capability of nand flash memory," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 937–955.

[4] X. Sun, H. Wan, Q. Li, C.-L. Yang, T.-W. Kuo, and C. J. Xue, "Rm-ssd: In-storage computing for large-scale recommendation inference," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1056–1070.

[5] S.-H. Tseng, T.-Y. Chen, and M.-C. Yang, "Are superpages super-fast? distilling flash blocks to unify flash pages of a superpage in an ssd," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 630–642.

[6] Y. Zhou, F. Wu, W. Huang, and C. Xie, "Livessd: A low-interference raid scheme for hardware virtualized ssds," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 7, pp. 1354–1366, 2020.

[7] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Bjørling, and H. S. Gunawi, "The {CASE} of {FEMU}: Cheap, accurate, scalable and extensible flash emulator," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 83–90.

[8] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "Improving 3d nand flash memory lifetime by tolerating early retention loss and process variation," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, pp. 1–48, 2018.

[9] D. Hong, M. Kim, J. Park, M. Jung, and J. Kim, "Improving ssd performance using adaptive restricted-copyback operations," in *2019 IEEE Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. IEEE, 2019, pp. 1–6.

[10] F. Wu, J. Zhou, S. Wang, Y. Du, C. Yang, and C. Xie, "Fastgc: Accelerate garbage collection via an efficient copyback-based data migration in ssds," in *Proceedings of the 55th Annual Design Automation Conference*, 2018, pp. 1–6.

[11] C. Ji, L. Wang, Q. Li, C. Gao, L. Shi, C.-L. Yang, and C. J. Xue, "Fair down to the device: A gc-aware fair scheduler for ssd," in *2019 IEEE Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. IEEE, 2019, pp. 1–6.

[12] N. Shahidi, M. T. Kandemir, M. Arjomand, C. R. Das, M. Jung, and A. Sivasubramaniam, "Exploring the potentials of parallel garbage collection in ssds for enterprise storage systems," in *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 561–572.

[13] Y. Zhou, E. Xu, L. Zhang, K. Karkra, M. Barczak, W. Gao, W. Malikowski, M. Kozlowski, Ł. Łasek, R. Lu *et al.*, "Csal: the next-gen local disks for the cloud," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 608–623.

[14] D. Lee, D. Hong, W. Choi, and J. Kim, "Mqsim-e: An enterprise ssd simulator," *IEEE Computer Architecture Letters*, vol. 21, no. 1, pp. 13–16, 2022.

[15] Q. Li, H. Dang, Z. Wan, C. Gao, M. Ye, J. Zhang, T.-W. Kuo, and C. J. Xue, "Midas touch: Invalid-data assisted reliability and performance boost for 3d high-density flash," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 657–670.

[16] S. Wang, F. Wu, Z. Lu, Y. Zhou, Q. Xiong, M. Zhang, and C. Xie, "Lifetime adaptive ecc in nand flash page management," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. IEEE, 2017, pp. 1253–1556.

[17] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu, "{MQSim}: A framework for enabling realistic studies of modern {Multi-Queue}{SSD} devices," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 49–66.

[18] J. Kim, M. Jung, and J. Kim, "Decoupled ssd: Rethinking ssd architecture through network-based flash controllers," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[19] D. Huang, D. Feng, Q. Liu, B. Ding, W. Zhao, X. Wei, and W. Tong, "A read latency variation aware independent read scheme for qlc ssds," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.

[20] N. Shibata, K. Kanda, T. Shimizu, J. Nakai, O. Nagao, N. Kobayashi, M. Miakashi, Y. Nagadomi, T. Nakano, T. Kawabe *et al.*, "A 1.33-tb 4-bit/cell 3-d flash memory on a 96-word-line-layer technology," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, pp. 178–188, 2019.

[21] B. Kim, S. Lee, B. Hah, K. Park, Y. Park, K. Jo, Y. Noh, H. Seol, H. Lee, J. Shin, S. Choi, Y. Jung, S. Ahn, Y. Park, S. Oh, M. Kim, S. Kim, H. Park, T. Lee, H. Won, M. Kim, C. Koo, Y. Choi, S. Choi, S. Park, D. Youn, J. Lim, W. Park, H. Hur, K. Kwean, H. Choi, W. Jeong, S. Chung, J. Choi, and S. Cha, "28.2 a high-performance 1tb 3b/cell 3d-nand flash with a 194mb/s write throughput on over 300 layers i," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 27–29.

[22] C. Gao, L. Shi, C. J. Xue, C. Ji, J. Yang, and Y. Zhang, "Parallel all the time: Plane level parallelism exploration for high performance ssds," in *2019 35th Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2019, pp. 172–184.

[23] C.-Y. Liu, Y. Lee, W. Choi, M. Jung, M. T. Kandemir, and C. Das, "Gssa: A resource allocation scheme customized for 3d nand ssds," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 426–439.

[24] S. Shadley, "NAND flash media management through RAIN." [Online]. Available: https://www.micron.com/-/media/client/global/documents/products/technical-marketing-brief/brief_ssd_rain.pdf

[25] R. Pletka, I. Koltsidas, N. Ioannou, S. Tomić, N. Papandreou, T. Parnell, H. Pozidis, A. Fry, and T. Fisher, "Management of next-generation nand flash to achieve enterprise-level endurance and latency targets," *ACM Transactions on Storage (TOS)*, vol. 14, no. 4, pp. 1–25, 2018.

[26] M. Kang, W. Lee, and S. Kim, "Subpage-aware solid state drive for improving lifetime and performance," *IEEE Transactions on Computers*, vol. 67, no. 10, pp. 1492–1505, 2018.

[27] C.-Y. Liu, J. B. Kotra, M. Jung, M. T. Kandemir, and C. R. Das, "Soml read: Rethinking the read operation granularity of 3d nand ssds," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 955–969.

[28] J. Kim, S. Kang, Y. Park, and J. Kim, "Networked ssd: Flash memory interconnection network for high-bandwidth ssd," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 388–403.

[29] R. Nadig, M. Sadrosadati, H. Mao, N. M. Ghiasi, A. Tavakkol, J. Park, H. Sarbazi-Azad, J. G. Luna, and O. Mutlu, "Venice: Improving solid-state drive parallelism at low cost via conflict-free accesses," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.

[30] C. Gao, X. Xin, Y. Lu, Y. Zhang, J. Yang, and J. Shu, "Parabit: processing parallel bitwise operations in nand flash memory based ssds," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 59–70.

[31] A. Khakifirooz, S. Balasubrahmanyam, R. Fastow, K. H. Gaewsky, C. W. Ha, R. Haque, O. W. Jungroth, S. Law, A. S. Madraswala, B. Ngo *et al.*, "30.2 a 1tb 4b/cell 144-tier floating-gate 3d-nand flash memory with 40mb/s program throughput and 13.8 gb/mm 2 bit density," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 424–426.

[32] J. Li, Q. Wang, P. P. Lee, and C. Shi, "An in-depth analysis of cloud block storage workloads in large-scale production," in *2020 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2020, pp. 37–47.